PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

Financial Transaction Analytics System

AN INTERNSHIP REPORT

Submitted by

BINDU N – 20221CSE0519

Under the guidance of,

Ms. Attar Mahay Sheetal

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Financial Transaction Analytics System"** is a Bonafide work of **BINDU N (20221CSE0519)** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

Ms. Attar Mahay Sheetal	**Mr. Sakthi S**	**Dr. Md Ziaur Rahaman**
Project Guide	Program Internship Coordinator	School Internship Coordinator
PSCS	PSCS	PSCS
Presidency University	Presidency University	Presidency University

Dr. Asif Mohammed H B	**Dr. Duraipandian N**
Head of the Department,	Dean
PSCS	PSCS / PSIS
Presidency University	Presidency University

Name and Signature of the Examiners

Sl. No	Name	Signature	Date
1			
2			

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING at Presidency University, Bengaluru, named BINDU N, hereby declare that the internship work titled **"Financial Transaction Analytics System"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

BINDU N USN: 20221CSE0519 Signature

PLACE: BENGALURU

DATE: 22 April 2026

INTERNSHIP COMPLETION CERTIFICATE

BNP PARIBAS

The bank for a changing world

Strictly Private & Confidential

Bindu N

December 05, 2025

Dear **Bindu**,

Internship Letter ("Agreement")

We are pleased to offer you a position as a Student Intern with **IT Department** of BNP Paribas India Solutions Pvt. Ltd (**"BNPP"**) in India on the terms and conditions set out in this Agreement.

For the purposes of this Agreement, an "**affiliate**" shall include any subsidiary or holding company of BNP Paribas.

1. Period of Internship

The term of your internship will be for a period of **6 months**, commencing on **January 13, 2026** and ending on **July 13, 2026**. For the avoidance of doubt, the terms and conditions of internship set out herein will be effective from the date you commence your internship with BNPP.

2. Internship Allowance

2.1 You will receive an internship allowance of **INR 50,000** (less applicable deductions) per calendar month (pro-rated for any incomplete month of internship). You acknowledge and agree that you will not receive any other benefit (in cash or in kind) from BNPP.

2.2 You may be permitted, at the discretion of BNPP, to be absent from your internship for urgent personal reasons for a period not exceeding 10 working days, and no deduction from your internship allowance will be made provided you have obtained prior written permission from BNPP for such absence. For the avoidance of doubt, deduction from your internship allowance will be made in respect of any unauthorized absence.

3. Place of Internship

Your place of internship will be at our offices in **Bengaluru** or such other location(s) within India as BNPP may require from time to time. During the course of your internship with BNPP, you may be assigned to any current or future divisions or departments of BNPP or its affiliates. You may also be required to travel within India or overseas as BNPP may request from time to time.

4. Hours of Internship

4.1 Your core internship hours are from 9:30 a.m. to 6:00 p.m., Monday to Friday (including half an hour lunch break). Your normal rest day will be Sunday (or such other day scheduled as a rest day). Saturday will normally be a contractual day off (unless an alternative contractual day off is scheduled). BNPP reserves the right to adjust the internship hours at its discretion.

4.2 From time to time you may be required to perform your internship duties outside the abovementioned core hours in order to meet our business needs and your individual objectives. You acknowledge and agree that you will not receive additional payment for any internship duties you perform outside of the abovementioned core hours.

BNP PARIBAS INDIA SOLUTIONS PRIVATE LIMITED
CIN: U72200MH2005PTC151511
Regd. Office: Block B3, 9th Floor, Nirlon Knowledge Park, Village Pahadi, Goregaon (E), Mumbai 400 063, INDIA. Tel : +91 22 6521 4000
Email : ispl.india@asia.bnpparibas.com Website Address: https://indiasolutions.bnpparibas.com

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Ms. Attar Mahay Sheetal, Assistant Professor,** Presidency School of Computer Science and Engineering, Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Asif Mohammed H B, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Mr. Sakthi S,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

BINDU N

ABSTRACT

In today's digital financial ecosystem, banking institutions process a high volume of transactions through platforms such as online banking, mobile applications, and payment systems. Efficient analysis of this data is essential for identifying transaction patterns, improving operational performance, and enabling informed decision-making. However, traditional data analysis approaches are often time-consuming and struggle to deliver timely insights in rapidly changing environments. This project presents a Financial Transaction Analytics System that analyzes and visualizes transaction data in near real time, transforming raw data into meaningful and actionable insights through an interactive dashboard. The system is composed of modules for data ingestion, processing, storage, analytics, and visualization, ensuring a smooth flow of data from source to insight generation. It produces key performance indicators such as total transaction count, transaction value trends, average transaction amounts, peak activity periods, and success versus failure rates. The dashboard provides an intuitive interface with dynamic charts and filtering options, allowing users to explore data effectively and gain a deeper understanding of transaction behavior. Overall, the system improves the efficiency of transaction monitoring, supports data-driven decision-making, and offers a scalable foundation for future enhancements, including real-time streaming integration and advanced analytics.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Internship completion certificate	iv
	Acknowledgement	v
	Abstract	vi
	List of Figures	ix
	List of Tables	x
	Abbreviations	xi
1.	Introduction	
	1.1 Background	
	1.2 Statistics	
	1.3 Prior existing technologies	
	1.4 Proposed approach	
	1.5 Objectives	
	1.6 SDGs	
	1.7 Overview of internship report	
2.	Literature review	
3.	Methodology	
4.	Project management	
	4.1 Internship timeline	
	4.2 Risk analysis	
5.	Analysis and Design	
	5.1 Requirements	
	5.2 Block Diagram	
	5.3 System Flow Chart	
	5.4 Standards	
	5.5 Domain model specification	
	5.6 Communication model	
	5.7 Functional view	
	5.8 Operational view	
6.	Software and Simulation	
	6.1 Software development tools	

6.2 Simulation
7. Evaluation and Results
7.1 Test points
7.2 Integration tests
7.3 Test result
7.4 Insights
8. Social, Legal, Ethical, Sustainability and Safety Aspects
9. Conclusion
References
Appendix

LIST OF FIGURES

Sl. No.	Figure	Caption	Page No.
1	Fig 1.1	Sustainable Development Goals	5
2	Fig 3.1	Methodology of Financial Transaction Analytics System	9
3	Fig 4.1	Project Timeline	11
4	Fig 5.2	Functional Block Diagram	14
5	Fig 5.3	System Flow Chart	16
6	Fig 5.5	Domain Model	19
7	Fig 5.6	Communication Model	20
8	Fig 5.7	Functional View	22

LIST OF TABLES

Sl. No	Table	Caption	Page No.
1	Table 2.1	Summary of Literature Reviews	8
2	Table 5.1	Description of Domain model	18

ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
BI	Business Intelligence
DB	Database
ETL	Extract, Transform, Load
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
REST	Representational State Transfer
JSON	JavaScript Object Notation
SQL	Structured Query Language
KPI	Key Performance Indicator
UI	User Interface
UX	User Experience
Kafka	Apache Kafka
MySQL	MySQL Database
Spring Boot	Spring Boot Framework

Chapter 1

Introduction

The rapid growth of digital banking and online payment systems has led to a significant increase in the volume of financial transactions generated every day. Financial institutions need efficient systems to monitor and analyze this data in order to understand transaction patterns, track performance, and support better decision-making. Traditional methods of data analysis are often slow and do not provide timely insights, making it difficult to handle large-scale transaction data effectively.

The Financial Transaction Analytics System is developed to address this need by processing and visualizing transaction data through an interactive dashboard. The system provides key insights such as transaction trends, total volume, average transaction value, and performance metrics. It uses technologies such as HTML, CSS, JavaScript, Java Spring Boot, MySQL, Chart.js, and Apache Kafka to build a scalable, efficient, and user-friendly analytics platform for financial data monitoring.

1.1 Background

With the advancement of digital technologies, the banking and financial sector has undergone significant transformation. The adoption of online banking, mobile payments, and digital wallets has resulted in a rapid increase in the volume and complexity of financial transactions. Managing this large amount of data has become a critical challenge for financial institutions, as it requires efficient systems to process, store, and analyze transaction information.

Earlier, financial data was analyzed using traditional methods that involved batch processing and manual reporting. These methods were time-consuming and did not provide real-time insights, limiting their usefulness in fast-paced environments. With the emergence of big data technologies and real-time processing systems, organizations can now analyze data more efficiently and gain insights almost instantly.

Data analytics and visualization have become essential tools in modern financial systems. By converting raw transaction data into meaningful insights through charts and dashboards, organizations can better understand transaction patterns, monitor system performance, and identify trends.

This project is based on these advancements and aims to develop a Financial Transaction Analytics System that utilizes modern technologies to process and visualize transaction data effectively. It builds upon the concepts of data analytics, real-time processing, and dashboard visualization to provide a practical solution for financial data monitoring.

1.2 Statistics

The banking and financial industry has witnessed rapid growth in digital transactions due to the increasing adoption of online banking, mobile applications, and digital payment systems. During the internship period, it was observed that financial institutions handle a very high volume of transactions on a daily basis, often reaching thousands to millions of transactions depending on the scale of operations. This highlights the importance of efficient systems for managing and analyzing transaction data.

Studies and industry reports indicate that digital payments are growing at a significant rate each year, with a large percentage of transactions now being carried out electronically. This growth has increased the demand for systems that can process and analyze transaction data quickly and accurately. Organizations are increasingly relying on data analytics to monitor transaction trends, measure performance, and improve operational efficiency.

Additionally, the use of real-time data processing technologies has become more prominent in modern financial systems. Systems are expected to handle high throughput with low latency to ensure timely insights. Visualization tools and dashboards are widely used to present transaction data in a meaningful way, allowing users to interpret complex datasets easily.

These observations during the internship emphasize the need for a Financial Transaction Analytics System that can efficiently process large volumes of data and provide real-time insights. The increasing scale and complexity of financial transactions make such analytics systems essential for modern banking operations.

1.3 Prior existing technologies

Earlier, financial institutions relied on traditional methods such as batch processing and manual reporting to analyze transaction data. In these systems, data was collected over a period of time and processed at scheduled intervals, which resulted in delays in obtaining insights. Database management systems like MySQL and PostgreSQL were used to store large volumes of transaction data, but they mainly focused on storage rather than real-time analysis. Business Intelligence tools such as Tableau and Power BI were later introduced to generate reports and dashboards, improving data visualization and decision-making.

With advancements in technology, modern systems have started using real-time data processing and scalable application frameworks. Technologies like Apache Kafka enable high-throughput data streaming, allowing systems to process transaction data continuously. Backend frameworks such as Java Spring Boot support efficient data processing and API development, while frontend technologies like HTML, CSS, JavaScript, and Chart.js help create interactive dashboards. However, many existing systems still lack seamless integration of real-time processing and analytics, which this project aims to address by combining these technologies into a unified solution.

1.4 Proposed approach

The proposed approach for the Financial Transaction Analytics System focuses on developing a scalable and efficient platform to process and visualize transaction data in near real time. The system collects transaction data from a dataset or a simulated real-time source and processes it through a backend application built using Java Spring Boot. Apache Kafka is used to simulate real-time data streaming, ensuring that the system can handle continuous data flow with high throughput. The processed data is then stored in a MySQL database for efficient retrieval and further analysis.

The analytics component of the system calculates key metrics such as total transactions, transaction value trends, average transaction amount, peak activity periods, and success rates. These insights are presented through an interactive dashboard developed using HTML, CSS, and JavaScript, with Chart.js used for data visualization. The system follows a modular architecture consisting of data ingestion, processing, storage, analytics, and visualization layers, making it scalable and easy to extend.

1.5 Objectives

The main objective of the Financial Transaction Analytics System is to design and develop a platform that can efficiently analyze and visualize financial transaction data. The system aims to provide meaningful insights that help in understanding transaction patterns and improving decision-making.

The specific objectives of the project are as follows:

- To collect and process financial transaction data from a dataset or simulated real-time source.
- To design a scalable backend system using Java Spring Boot for data processing and API handling.
- To store and manage transaction data efficiently using MySQL.
- To analyze transaction data and generate key metrics such as total transactions, average transaction value, and transaction trends.
- To develop an interactive dashboard using HTML, CSS, JavaScript, and Chart.js for data visualization.
- To simulate real-time data streaming using Apache Kafka.
- To provide a user-friendly interface for monitoring and analyzing transaction data.
- To ensure the system is scalable, efficient, and easy to extend for future enhancements.

1.6 SDG

Fig 1.1 Sustainable development goals

The Financial Transaction Analytics System aligns with several United Nations Sustainable Development Goals (SDGs) by promoting the effective use of technology and supporting economic growth through data-driven systems.

This project contributes to SDG 8: Decent Work and Economic Growth by enabling financial institutions to improve operational efficiency and decision-making through data analytics. By providing insights into transaction patterns and performance, the system helps organizations optimize processes and enhance productivity.

It also supports SDG 9: Industry, Innovation, and Infrastructure by utilizing modern technologies such as real-time data processing, scalable backend systems, and interactive dashboards. The project demonstrates how innovative digital solutions can strengthen financial infrastructure and support the development of intelligent systems for data analysis

1.7 Overview of internship report

This report consists of nine chapters that describe the design and development of the Financial Transaction Analytics System. Chapter 1 introduces the project by presenting the background, objectives, need for transaction analytics, existing technologies, alignment with sustainable development goals, and overall project goals. Chapter 2 presents the literature review, which

discusses previous research in data analytics, real-time data processing, financial systems, and dashboard visualization techniques.

Chapter 3 explains the methodology used in the project, including data collection, data processing, analytics generation, and system architecture. Chapter 4 focuses on project management aspects such as internship timeline, risk analysis, and project planning. It also provides an overview of how the project was organized and executed during the internship period.

Chapter 5 describes the system analysis and design, including requirements, block diagrams, system flow charts, domain models, communication models, and overall system architecture. Chapter 6 presents the implementation details, including software tools, technologies used, and development approach for building the system.

Chapter 7 discusses the evaluation and results of the system, including testing strategies, performance analysis, and insights obtained from the analytics dashboard. Chapter 8 highlights the social, legal, ethical, sustainability, and safety aspects associated with the project, focusing on responsible data usage and system impact. Finally, Chapter 9 concludes the report by summarizing the key findings and suggesting possible future enhancements for the system.

Chapter 2

Literature review

The rapid growth of digital transactions has increased the need for efficient data analytics systems in the financial sector. Business intelligence and analytics play a significant role in transforming raw transaction data into meaningful insights that support decision-making. Chen et al. [1] emphasized the importance of data-driven systems in organizations and highlighted how analytics improves operational efficiency and business performance.

Real-time data processing has become essential for handling large volumes of transaction data. Kreps et al. [2] introduced Apache Kafka as a distributed messaging system capable of processing high-throughput data streams with low latency. Similarly, Akidau et al. [3] proposed the Dataflow model, which provides a unified approach for processing both batch and streaming data efficiently while maintaining correctness and performance.

Big data analytics techniques further enhance the ability to process and analyze massive datasets. Gandomi and Haider [4] discussed the importance of scalable analytics frameworks for extracting valuable insights from complex data. These techniques are particularly useful in financial systems where large-scale transaction data must be analyzed efficiently.

Visualization plays a crucial role in analytics systems by helping users interpret complex data. Heer et al. [5] explored various visualization techniques and demonstrated how graphical representations improve data understanding. Few [6] highlighted the importance of dashboard design in presenting key performance indicators in a clear and effective manner.

In the financial domain, analyzing transaction data is important for identifying patterns and understanding system behavior. Bhattacharyya et al. [7] demonstrated how data mining techniques can be applied to financial data for pattern extraction. Additionally, research on stream processing systems [8] highlights the importance of scalability, fault tolerance, and performance in handling continuous data streams.

Overall, existing studies show that combining real-time data processing, scalable system architecture, and effective visualization techniques is essential for building efficient transaction analytics systems. These concepts form the foundation for the proposed Financial Transaction Analytics System.

Summary of Literatures reviewed

Table 2.1 Summary of Literature review

Author / Reference	**Key Contribution**
Chen et al. [1]	Highlighted the importance of business intelligence and analytics in improving decision-making and organizational performance.
Kreps et al. [2]	Introduced Apache Kafka for real-time data streaming with high throughput and low latency.
Akidau et al. [3]	Proposed the Dataflow model for efficient batch and stream processing with accuracy and performance.
Gandomi & Haider [4]	Discussed big data analytics techniques for processing large-scale and complex datasets.
Heer et al. [5]	Explained visualization techniques to improve data understanding through graphical representation.
Few [6]	Emphasized effective dashboard design for presenting key performance indicators clearly.
Bhattacharyya et al. [7]	Demonstrated the use of data mining techniques for analyzing financial transaction data and extracting patterns.
Stream Processing Studies [8]	Highlighted the importance of scalability, fault tolerance, and performance in real-time data processing systems.

Chapter 3

Methodology

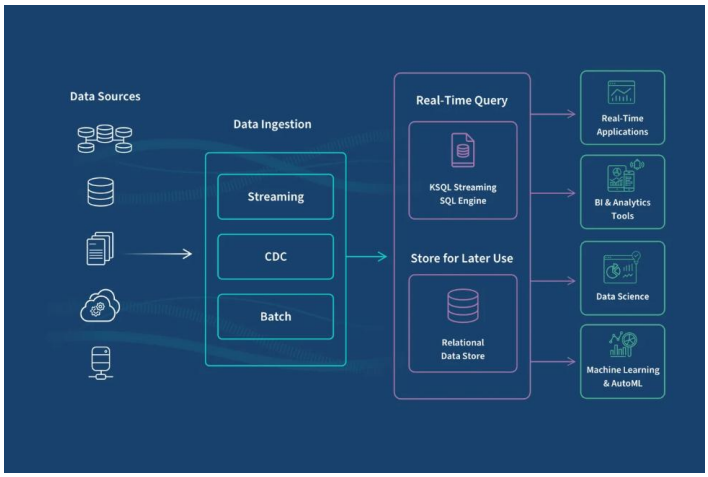

Fig 3.1 Methodology of Financial Transaction Analytics System

The methodology of the Financial Transaction Analytics System follows a structured pipeline approach, as shown in the methodology flow diagram. The system is designed to handle transaction data efficiently by processing it through multiple stages, including data ingestion, processing, storage, analytics, and visualization. This approach ensures a smooth flow of data and enables the generation of meaningful insights.

In the first stage, transaction data is collected from a dataset or generated through simulated real-time streaming. Apache Kafka is used to handle the data ingestion process, allowing continuous and high-throughput data flow into the system. This helps in simulating real-world financial environments where large volumes of transactions are processed continuously. Kafka also ensures reliable message delivery and acts as a buffer between data producers and consumers.

The second stage involves data processing, where the incoming data is cleaned, transformed, and structured. This step ensures that the data is consistent and ready for analysis. The backend, developed using Java Spring Boot, manages this processing and acts as a bridge between the data source and the storage system. It also handles API requests, validation, error handling, and system logic. Additional steps such as parsing and filtering are performed to ensure only relevant and accurate data is processed further.

In the third stage, the processed data is stored in a MySQL database. This allows efficient storage and quick retrieval of transaction records. Proper database design techniques such as indexing and normalization are applied to improve performance. The stored data is then used by the analytics module to compute key metrics such as transaction trends, system performance indicators, and activity patterns. These analytics help in identifying patterns and understanding system behavior.

The fourth stage focuses on analytics, where the system processes stored data to derive meaningful insights. This includes generating summaries, identifying trends over time, and categorizing transactions based on different parameters. The analytics module ensures that data is processed efficiently to support real-time or near real-time insights.

Finally, the visualization stage presents the analyzed data through an interactive dashboard. The frontend is developed using HTML, CSS, and JavaScript, while Chart.js is used to create graphical representations such as charts and graphs. The dashboard provides an intuitive interface with filtering options, enabling users to explore data easily and gain insights. It also supports dynamic updates, allowing users to view the latest processed data.

Overall, this methodology ensures a scalable, modular, and efficient system architecture. Each stage is designed to handle specific tasks, making the system easy to maintain and extend. The integration of real-time streaming, backend processing, database management, and visualization provides a complete solution for financial transaction analytics.

Chapter 4

Project Management

4.1 Internship timeline

The internship was carried out over a period of six months, during which the initial phase focused on understanding the existing system and gradually contributing to its development and enhancement. Since the Financial Transaction Analytics System was already in place, the work involved learning, supporting, and improving different components of the system.

- Month 1–3: Understanding the existing system, architecture, and workflows. This phase involved gaining knowledge of the financial transaction analytics platform, understanding the technology stack, exploring the codebase, and learning about data flow, dashboards, and system functionalities.
- Month 4: Requirement understanding and identifying areas for improvement or enhancement in the existing system. This included analyzing system behavior and discussing tasks with the team.
- Month 5: Development and implementation of assigned features or improvements using technologies such as Java Spring Boot, MySQL, and frontend tools.
- Month 6: Testing, debugging, performance evaluation, and documentation of the implemented changes and system understanding.

Fig 4.1 Project Timeline

4.2 Risk analysis

Risk analysis is an important part of the project to identify potential challenges that may affect the development and performance of the Financial Transaction Analytics System. Since the project involves working on an existing system, risks are mainly related to understanding the system, handling data, and implementing changes without affecting current functionality.

One of the major risks is the complexity of the existing system. As the system is already developed and in use, understanding the architecture, codebase, and data flow can be challenging and time-consuming. To reduce this risk, sufficient time was allocated during the initial phase for learning and documentation.

Another risk is related to data handling and accuracy. Incorrect or incomplete data can affect the analytics results and lead to misleading insights. This can be minimized by ensuring proper data validation, cleaning, and testing during development.

There is also a risk of integration issues when adding new features or modifying existing components. Changes in one part of the system may impact other modules. This risk can be managed by following proper testing practices, maintaining modular design, and performing thorough debugging.

Performance-related risks may arise when handling large volumes of transaction data, which can affect system speed and responsiveness. This can be addressed by optimizing database queries, improving backend processing, and using efficient data handling techniques.

Finally, time constraints during the internship period can impact the completion of tasks. Proper planning, prioritization of tasks, and regular progress tracking help in managing this risk effectively.

Overall, identifying and addressing these risks ensures smooth development and successful implementation of the project.

Chapter 5
Analysis and Design

The Analysis and Design phase focuses on understanding the system requirements and creating a structured plan for developing the Financial Transaction Analytics System. This phase plays a crucial role in defining how the system will function, how different components will interact, and how data will flow through the system.

In the analysis stage, the existing system was studied to understand its architecture, data flow, and functionalities. Requirements were identified based on system behavior, user needs, and improvements required in the analytics dashboard. This included identifying the type of transaction data, key metrics to be generated, and the overall objectives of the system.

In the design stage, the system architecture was planned to ensure scalability, efficiency, and modularity. The system was divided into components such as data ingestion, processing, storage, analytics, and visualization. Diagrams such as block diagrams and flowcharts were used to represent the system structure and data flow. The design also included defining communication between frontend and backend, database structure, and user interface layout.

Overall, this phase ensures that the system is well-structured and clearly defined before implementation. It helps in reducing development errors, improving system performance, and ensuring that the final system meets all functional and non-functional requirements.

5.1 Requirements

The requirements of the Financial Transaction Analytics System define the functionalities and conditions needed for the system to operate effectively. These requirements are categorized into functional and non-functional requirements.

Functional requirements describe what the system should do. The system should be able to collect transaction data from a dataset or streaming source. It should process and store the data efficiently in a database. The system must analyze the data to generate key metrics such as total transactions, transaction trends, average transaction value, and success rates. It should display the analyzed data through an interactive dashboard with charts and graphs. The system should also provide filtering options to view data based on time range or categories.

Non-functional requirements define how the system should perform. The system should be scalable to handle large volumes of transaction data. It should provide fast response time to ensure smooth user experience. The system must be reliable and ensure data accuracy during processing and visualization. Security should be maintained to protect transaction data. The system should also be user-friendly, allowing users to easily navigate and interpret the dashboard.

These requirements ensure that the system is efficient, reliable, and capable of providing meaningful insights from financial transaction data.

5.2 Block diagram

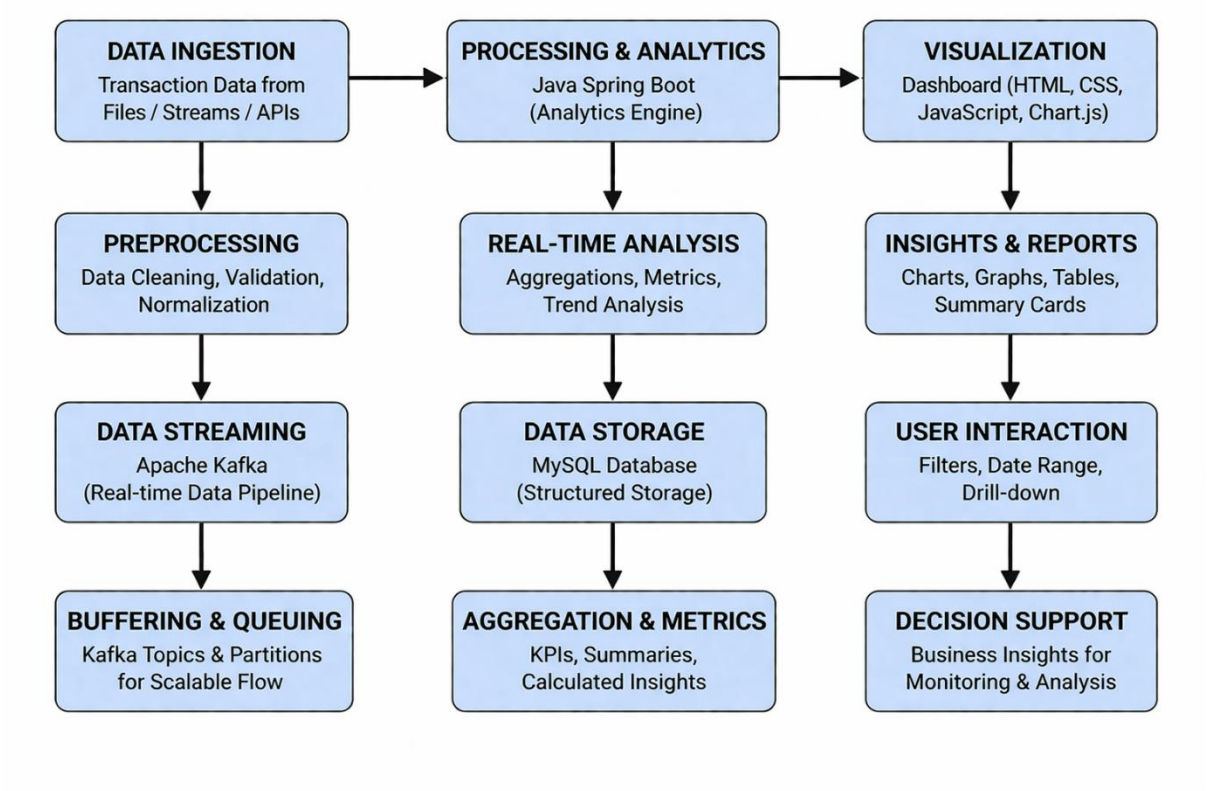

Fig 5.2 Functional block diagram

The block diagram of the Financial Transaction Analytics System illustrates the overall architecture and flow of data across different components of the system. It provides a clear understanding of how transaction data is collected, processed, stored, analyzed, and finally visualized for user interaction.

The process begins with the data sources, where transaction data is obtained from datasets or real-time streams. This data is passed to the data ingestion module, which acts as the entry

point of the system. The ingestion layer ensures that data is properly received and forwarded for further processing.

The next stage involves streaming and processing, where Apache Kafka is used to handle real-time data flow. This component ensures high-throughput data streaming and enables the system to process continuous transaction data efficiently. The data is then sent to the database integration module, where it is stored in a structured format using MySQL for efficient retrieval and management.

Following storage, the analysis module processes the stored data using Java Spring Boot. This module performs calculations and generates key insights such as transaction trends, total transaction values, and performance metrics. The analyzed data is then passed to the dashboard and visualization module.

The dashboard is developed using HTML, CSS, JavaScript, and Chart.js, which presents the data in the form of charts, graphs, and tables. Users can interact with the dashboard to explore data through filters and gain meaningful insights. Finally, the system outputs results and insights that support monitoring and decision-making.

Overall, the block diagram demonstrates a structured and scalable system architecture, ensuring efficient data flow from input to visualization.

5.3 System Flow chart

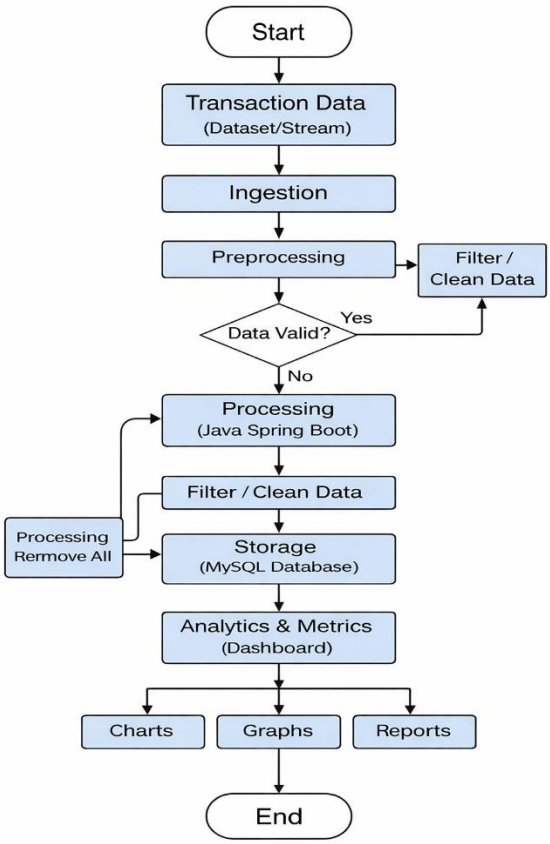

Fig 5.3 System flow chart

The system flow chart illustrates the step-by-step process followed in the Financial Transaction Analytics System, starting from data input to the generation of final insights. It provides a clear understanding of how transaction data moves through different stages of the system.

The process begins with the start node, followed by the input of transaction data, which may be obtained from a dataset or a real-time stream. This data is then passed to the ingestion stage, where it is collected and forwarded for further processing. In the preprocessing stage, the data is cleaned, validated, and transformed to ensure consistency and accuracy.

A decision step is included to check whether the data is valid. If the data is not valid, it is filtered and cleaned again before being processed. If the data is valid, it moves to the processing stage, where the backend system built using Java Spring Boot performs computations and prepares the data for storage.

The processed data is then stored in the MySQL database, ensuring structured and efficient data management. After storage, the analytics module calculates key metrics and trends from the data. These results are then displayed on the dashboard, where they are visualized in the form of charts, graphs, and reports.

Finally, the process ends with the user viewing and interacting with the dashboard to gain insights. This flow chart clearly represents the logical sequence of operations in the system and helps in understanding how data is transformed into meaningful analytics.

5.4 Standards

The development of the Financial Transaction Analytics System follows standard practices and guidelines to ensure quality, reliability, and maintainability of the system. These standards help in creating a structured and efficient application that can be easily understood, tested, and extended.

Coding standards are followed during development to maintain consistency and readability of the code. Java coding conventions are used for backend development with Spring Boot, while proper naming conventions and modular structure are maintained in frontend technologies such as HTML, CSS, and JavaScript. This ensures that the codebase is clean and easy to manage.

Database standards are applied in designing the MySQL database, including proper normalization, use of primary and foreign keys, and efficient indexing. These practices help in maintaining data integrity and improving query performance.

API design standards are followed while developing backend services. RESTful APIs are used to enable communication between frontend and backend, ensuring scalability and flexibility. Proper request and response formats are maintained for consistency.

Security standards are considered to protect transaction data. This includes secure data handling, validation of inputs, and controlled access to the system. Additionally, performance and scalability standards are maintained to ensure that the system can handle large volumes of transaction data efficiently.

Overall, following these standards ensures that the system is robust, maintainable, and aligned with industry best practices.

5.5 Domain model specification

The domain model diagram represents the key entities of the Financial Transaction Analytics System and shows how they are related to each other. It provides a clear understanding of how data flows between different components and how the system is structured at a conceptual level.

The User entity represents the customer or account holder in the system. Each user can have one or more accounts, which is represented by the relationship between the User and Account entities. The Account entity contains details such as account number, balance, and account type. This shows that a single user may manage multiple bank accounts.

The Transaction entity is the central component of the system. Each account can have multiple transactions, and each transaction contains details such as transaction ID, amount, date and time, status, and type. Transactions are categorized using the Category entity, which helps in classifying transactions into types such as payment, transfer, or withdrawal. This classification supports better analysis of transaction data.

The Analytics entity is responsible for generating insights from the transaction data. It includes metrics such as total transactions, average transaction value, and transaction trends. These analytics are then connected to the Dashboard entity, which displays the results in the form of charts, graphs, and reports.

The relationships shown in the diagram, such as one-to-one, one-to-many, and many-to-one, indicate how entities are

Table 5.5 Description of Domain model

Domain Element	**Description**
Physical Entity	Financial transactions generated from banking systems, payment gateways, or datasets (input sources).
Virtual Entity	Processed and structured transaction data stored in the database for analysis.
Device	Servers and systems used to process transaction data, including backend servers and streaming platforms.

Resource	Software components such as Apache Kafka for streaming, Java Spring Boot for processing, MySQL for storage, and Chart.js for visualization.
Service	REST APIs and dashboard services that provide analytics, reports, and transaction insights to users.

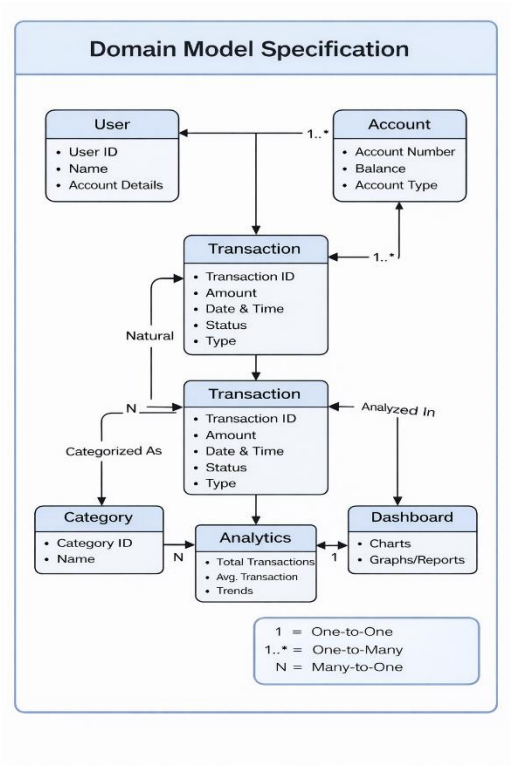

Fig 5.5 Domain model

5.6 Communication model

The communication model of the Financial Transaction Analytics System describes how different components of the system interact and exchange data with each other. It defines the flow of information between the data source, backend, database, and frontend dashboard. The communication begins with the data source, where transaction data is generated from datasets or real-time streams. This data is transmitted to the system through Apache Kafka, which acts as a messaging and streaming platform. Kafka ensures reliable and continuous data flow between components, enabling real-time data ingestion.

The backend, developed using Java Spring Boot, communicates with Kafka to consume incoming data streams. It processes the data, performs necessary transformations, and then interacts with the MySQL database to store and retrieve transaction information. The backend

also exposes REST APIs, which act as a communication bridge between the frontend and backend systems.

The frontend dashboard communicates with the backend through these REST APIs. It sends requests to fetch processed data and receives responses in a structured format such as JSON. The frontend then uses this data to display charts, graphs, and reports using Chart.js.

Overall, the communication model ensures smooth and efficient data exchange between all components of the system. It supports real-time data flow, reliable processing, and seamless interaction between the user interface and backend services.

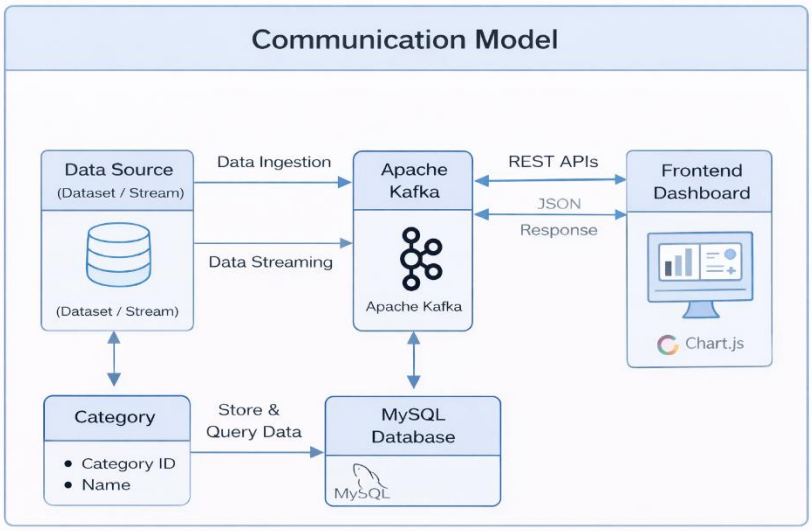

Fig 5.6 Communication Model of Financial Transaction Analytics System

Figure 5.6 The figure illustrates the communication model of the Financial Transaction Analytics System, showing how different components interact and exchange data within the system. It highlights the flow of transaction data from the source to the final dashboard through various intermediate components.

The process begins with the data source, where transaction data is generated from datasets or real-time streams. This data is sent to Apache Kafka through the data ingestion and streaming process. Kafka acts as a messaging system that ensures reliable and continuous data flow between components.

The backend system, developed using Java Spring Boot, consumes data from Kafka, processes it, and stores it in the MySQL database. The database is responsible for storing structured transaction data and supporting efficient querying and retrieval. Additional entities such as categories may also be used to classify and organize the data.

The frontend dashboard communicates with the backend using REST APIs. Data is transferred in JSON format, allowing smooth interaction between frontend and backend systems. The dashboard, built using Chart.js, displays the processed data in the form of charts, graphs, and reports.

Overall, the communication model ensures efficient data exchange, real-time processing, and seamless integration between system components, enabling users to visualize and analyze transaction data effectively.

5.7 Functional view

The functional view of the Financial Transaction Analytics System describes the major functions performed by the system and how different components work together to achieve the overall objective. It focuses on the logical operations of the system rather than the physical implementation.The system begins with the data ingestion function, where transaction data is collected from datasets or real-time streams. This data is then passed to the preprocessing and processing functions, where it is cleaned, validated, and transformed into a structured format suitable for analysis.

The next function is data storage, where the processed data is stored in the MySQL database. This allows efficient retrieval and management of transaction data. The analytics function then processes this stored data to generate key metrics such as total transactions, trends, and performance indicators.Finally, the visualization function presents the analyzed data through an interactive dashboard using charts and graphs. Users can interact with the system through filters and controls to explore insights.

Overall, the functional view highlights how different system functions are organized and how they collaborate to transform raw transaction data into meaningful insights.

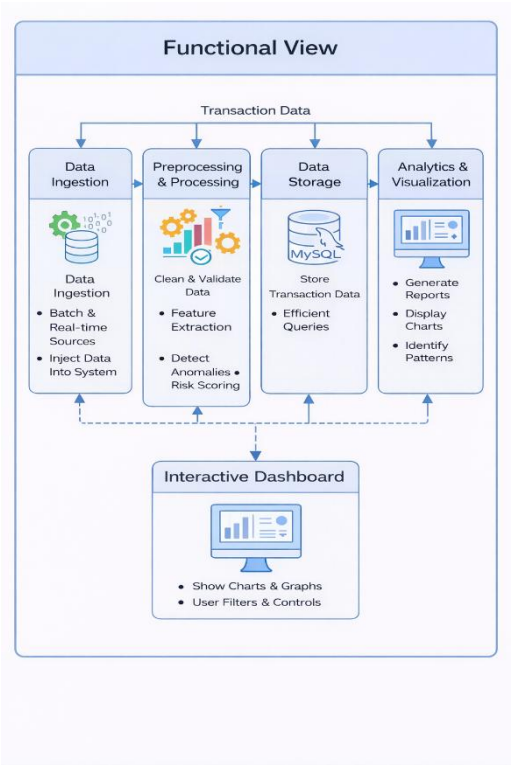

Fig 5.7 Functional view

5.8 Operational view

The operational view of the Financial Transaction Analytics System describes how the system functions in a real-time environment and how different components operate together during execution. It focuses on the runtime behavior of the system, including data flow, processing, and user interaction.

The operation begins with the generation of transaction data from data sources such as datasets or real-time streams. This data is continuously sent to the system through Apache Kafka, which handles data streaming and ensures reliable communication between components. Kafka manages high-throughput data flow and supports real-time processing requirements.

The backend system, developed using Java Spring Boot, consumes the incoming data from Kafka and performs data processing tasks such as validation, transformation, and preparation for storage. The processed data is then stored in the MySQL database, which enables efficient querying and retrieval of transaction records.

Once the data is stored, the analytics module processes the data to compute key metrics and trends. These insights are then made available to the frontend dashboard through REST APIs.

The frontend, built using HTML, CSS, JavaScript, and Chart.js, fetches the data and displays it in the form of charts, graphs, and reports.

From an operational perspective, the system runs continuously, handling incoming data, processing it, and updating the dashboard in near real time. It ensures smooth coordination between all components, providing users with up-to-date insights and enabling efficient monitoring of transaction data.

Chapter 6

Software and Simulation

6.1 Software development tools

The Financial Transaction Analytics System is developed using a set of modern software tools and technologies that support efficient development, data processing, and visualization. These tools are selected based on their scalability, performance, and ease of integration.
The backend of the system is developed using Java Spring Boot, which provides a robust framework for building scalable and efficient applications. It is used for handling data processing, business logic, and creating REST APIs for communication between different components of the system.
For frontend development, HTML, CSS, and JavaScript are used to design and build the user interface. Chart.js is used as a visualization library to create interactive charts and graphs, enabling users to easily interpret transaction data through the dashboard.
MySQL is used as the database management system to store and manage transaction data. It provides efficient data storage, retrieval, and supports structured query operations required for analytics.
Apache Kafka is used for handling real-time data streaming. It enables the system to process continuous data flow with high throughput and low latency, making it suitable for real-time transaction analytics.
Additionally, development tools such as an Integrated Development Environment (IDE) like IntelliJ IDEA or Eclipse are used for coding and debugging. Version control tools like Git are used to manage code changes and maintain project versions.
Overall, these tools work together to build a scalable, efficient, and user-friendly financial transaction analytics system.

6.2 Simulation

The simulation of the Financial Transaction Analytics System is carried out to replicate real-world transaction data flow and evaluate the system's performance under continuous data input. Since the project does not involve actual live banking data, a simulated environment is created where transaction data is generated either from datasets or through controlled streaming mechanisms. Apache Kafka is used to simulate real-time data streaming, allowing the system

to process a continuous flow of transaction records as it would in a real financial system. This helps in testing how efficiently the system handles high volumes of data and ensures that the data pipeline from ingestion to visualization functions smoothly.

During the simulation, transaction data is sent through the system pipeline, where it is processed by the backend developed using Java Spring Boot, stored in the MySQL database, and then analyzed to generate key metrics. The frontend dashboard, built using HTML, CSS, JavaScript, and Chart.js, displays the processed data in the form of charts and graphs. The simulation helps in verifying system behavior, identifying performance bottlenecks, and ensuring that all components such as data ingestion, processing, storage, and visualization are working correctly. It also provides a safe environment to test different scenarios, making it easier to validate the system before real-world deployment.

Chapter 7

Evaluation and Results

7.1 Test points

Test points define the key areas of the system that are verified to ensure the Financial Transaction Analytics System functions correctly and efficiently. These points help in evaluating whether each component of the system is working as expected.

The first test point is data ingestion, where the system is tested to ensure that transaction data is correctly received from the data source or simulated stream using Apache Kafka. The system should be able to handle continuous data flow without loss of data.

The second test point is data processing, where the backend developed using Java Spring Boot is tested for correct data cleaning, validation, and transformation. It is verified that the system processes data accurately and handles invalid or incomplete data properly.

The third test point is database storage, where the system is checked to ensure that processed data is correctly stored in the MySQL database. This includes verifying data integrity, proper structure, and efficient retrieval of records.

The fourth test point is analytics generation, where the system is tested to confirm that key metrics such as total transactions, average values, and trends are calculated correctly.

The fifth test point is dashboard visualization, where the frontend is tested to ensure that charts and graphs are displayed correctly using Chart.js. It is also verified that filters and user interactions work as expected.

The final test point is system performance, where the system is evaluated for response time, data processing speed, and ability to handle large volumes of transaction data.

These test points ensure that all components of the system are functioning correctly and meet the required performance standards.

7.2 Integration Tests

Integration testing is performed to ensure that all components of the Financial Transaction Analytics System work together seamlessly. Since the system consists of multiple modules

such as data ingestion, processing, storage, analytics, and visualization, it is important to verify the interaction between these components.

The first integration test focuses on data ingestion and backend processing. It verifies that transaction data received through Apache Kafka is correctly consumed by the Java Spring Boot application without data loss or delay.

The second integration test is parser testing, where the system checks whether incoming data is correctly parsed and interpreted. It ensures that fields such as transaction ID, amount, timestamp, and status are accurately extracted from the input format.

The third integration test is data transformation testing, which ensures that raw data is properly cleaned, validated, and converted into a structured format before storage. This step verifies that the transformation logic maintains data consistency and correctness.

The next integration test is database integration, where it is verified that processed data is correctly stored in the MySQL database and can be retrieved efficiently for further processing.

Another important test is analytics integration, which checks whether the analytics module correctly processes stored data and generates accurate metrics such as transaction counts, trends, and averages.

The frontend-backend integration test ensures that REST APIs function correctly and that the frontend dashboard receives accurate data from the backend. It also verifies that the data is properly displayed using Chart.js.

Finally, performance integration testing is conducted to evaluate how well the system handles continuous data flow and large volumes of transactions. It ensures that all components work together efficiently without performance degradation.

These integration tests ensure that the system operates as a unified solution, providing accurate and reliable transaction analytics.

7.3 Test Result

The testing of the Financial Transaction Analytics System was conducted to evaluate the correctness, performance, and integration of all system components. The results indicate that

the system performs efficiently and meets the expected functional and non-functional requirements.

The data ingestion module successfully received and processed continuous transaction data through Apache Kafka without any data loss. The parser correctly extracted required fields such as transaction ID, amount, and timestamp from the input data. The data transformation process ensured that the data was cleaned, validated, and structured properly before storage.

The backend processing using Java Spring Boot performed as expected, handling data efficiently and applying the required logic for analytics. The integration with the MySQL database was successful, with data being stored and retrieved accurately. No inconsistencies were observed in the stored data.

The analytics module generated correct results for key metrics such as total transactions, average transaction value, and transaction trends. The frontend dashboard successfully displayed these results using Chart.js, and all visualizations were accurate and responsive. User interactions such as filtering and data selection worked correctly.

Performance testing showed that the system handled large volumes of transaction data with acceptable response time and throughput. The integration between different modules was smooth, and no major issues were encountered during testing.

Overall, the test results confirm that the system is reliable, efficient, and capable of providing accurate transaction analytics through an interactive dashboard.

7.4 Insights

The Financial Transaction Analytics System provides valuable insights by analyzing transaction data and presenting it in a structured and visual format. Through the dashboard, users can clearly observe transaction trends over different time periods, helping in understanding peak activity hours and usage patterns.

The system highlights key metrics such as total number of transactions, total transaction value, and average transaction amount. These insights help in evaluating overall system performance and identifying growth patterns in transaction activity. Users can also analyze the distribution of transactions based on categories or status, such as successful and failed transactions.

Another important insight is the ability to identify variations in transaction behavior. By comparing data over time, users can observe changes in transaction frequency and volume,

which can be useful for performance monitoring and planning. The filtering options in the dashboard allow users to focus on specific time ranges or conditions, making the analysis more flexible and detailed.

Overall, the system transforms raw transaction data into meaningful insights that support better understanding and decision-making. These insights demonstrate the effectiveness of analytics and visualization in managing and monitoring financial transaction systems.

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

The Financial Transaction Analytics System has several important implications in terms of social, legal, ethical, sustainability, and safety aspects. These factors are considered to ensure that the system is responsible, secure, and beneficial to users and organizations.

From a social perspective, the system improves accessibility to financial insights by presenting complex transaction data in a simple and understandable format. It helps organizations make better decisions, which can indirectly improve customer experience and service quality. However, it is important to ensure that the system is inclusive and easy to use for all types of users.

In terms of legal aspects, the system must comply with data protection and privacy regulations. Financial data is highly sensitive, so proper measures must be taken to ensure secure data handling, storage, and access control. The system should follow relevant guidelines and standards to protect user information and prevent unauthorized access.

Ethically, the system must ensure responsible use of data. Transaction data should only be used for analysis and monitoring purposes, and not for misuse or unauthorized activities. Transparency in how data is processed and presented is important to maintain trust. Any analytics generated should be accurate and unbiased.

From a sustainability perspective, the system should be designed to use resources efficiently. Optimizing data processing and storage helps reduce computational load and energy consumption. Using scalable technologies ensures that the system can grow without unnecessary resource usage.

Safety aspects focus on ensuring system reliability and protection against failures. The system should be robust, with proper error handling and backup mechanisms to prevent data loss. Secure communication between components and protection against cyber threats are also important to maintain system safety.

Overall, considering these aspects ensures that the Financial Transaction Analytics System is not only functional but also responsible, secure, and sustainable.

Chapter 9

Conclusion

The Financial Transaction Analytics System developed during the internship demonstrates the effective use of modern technologies to analyze and visualize transaction data in a structured and meaningful manner. The system integrates key components such as data ingestion, processing, storage, analytics, and visualization to form a complete analytics pipeline. Using technologies like Apache Kafka, Java Spring Boot, MySQL, and Chart.js, it efficiently handles large volumes of data and presents insights through an interactive dashboard. The project emphasizes the importance of data analytics in the financial domain, where timely and accurate insights are critical for monitoring performance and understanding transaction patterns. It generates key metrics such as transaction trends, total values, and performance indicators, enabling improved decision-making and operational efficiency. The visualization features simplify complex data interpretation, making insights more accessible to users. The internship provided valuable hands-on experience with real-world systems, system architecture, and collaborative development practices, along with exposure to the full software development lifecycle. While the system performs effectively, there is potential for future enhancements such as advanced analytics, predictive modeling, improved real-time processing, role-based access, and stronger security measures. Overall, the system offers a scalable, efficient, and user-friendly solution, showcasing the role of data analytics and visualization in enhancing financial system performance and decision-making.

References

[1] Chen, H., Chiang, R. H. L., & Storey, V. C., "Business Intelligence and Analytics: From Big Data to Big Impact," MIS Quarterly, vol. 36, no. 4, pp. 1165–1188, 2012.

[2] Kreps, J., Narkhede, N., & Rao, J., "Kafka: A Distributed Messaging System for Log Processing," NetDB, 2011.

[3] Akidau, T., et al., "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale Data Processing," VLDB, 2015.

[4] Gandomi, A., & Haider, M., "Beyond the Hype: Big Data Concepts, Methods, and Analytics," International Journal of Information Management, vol. 35, no. 2, pp. 137–144, 2015.

[5] Heer, J., Bostock, M., & Ogievetsky, V., "A Tour Through the Visualization Zoo," Communications of the ACM, vol. 53, no. 6, pp. 59–67, 2010.

[6] Few, S., *Information Dashboard Design: The Effective Visual Communication of Data*, O'Reilly Media, 2006.

[7] Bhattacharyya, S., et al., "Data Mining for Credit Card Data: A Comparative Study," Decision Support Systems, vol. 50, no. 3, pp. 602–613, 2011.

[8] Zaharia, M., et al., "Discretized Streams: Fault-Tolerant Streaming Computation at Scale," ACM Symposium on Operating Systems Principles, 2013.

Appendix

i. Data Sheets: Since the project is fully software-driven, the system replaces physical sensors with virtual components such as Apache Kafka (for data streaming and ingestion), Java Spring Boot (for data processing and API handling), MySQL (for data storage), and Chart.js (for data visualization). These components act as digital processing units that collect, manage, analyze, and present financial transaction data. Together, they function as a unified analytics pipeline that continuously processes transaction data and generates meaningful insights through an interactive dashboard.

ii. GitHub Link

https://github.com/bindu-n323/Financial-Transaction-Analytics-System-.git

iii. Project Image

a. Dashboard Interface